Software Testing and Quality Assurance

1️⃣4️⃣ Approval

- QA Lead
- Project Manager

STEP 3: Write Sample Test Plan (Short Version)

You can write like this in exam:

Test Plan ID: TP_LMS_001

Objective:
To test Library Management System functionalities.

Scope:
Includes login, book issue, return.

Test Strategy:
Manual + Automation testing.

Environment:
Windows, Chrome, Selenium.

Entry Criteria:
Application ready.

Exit Criteria:
All test cases passed.

Deliverables:
Test plan, test cases, bug report.

OUTPUT

✓ Complete **Test Plan Document**
✓ Proper headings
✓ Structured format

Practical No-6

Selenium IDE – Create Test Case & Test Suite

Aim

To create and execute **test cases and test suite using Selenium IDE**.

Software Testing and Quality Assurance

Software/Tools Required

- Mozilla Firefox / Chrome
- Selenium IDE Extension
- Internet connection

PART 1: INSTALLATION OF SELENIUM IDE

◇ **Step 1: Install Selenium IDE**

Steps:

1. Open **Chrome / Firefox browser**
2. Go to Extensions:
    - Chrome → Chrome Web Store
3. Search: **Selenium IDE**
4. Click **Add to Chrome / Add to Firefox**
5. Install extension

👉 After installation, Selenium IDE icon appears in toolbar

PART 2: MANUAL TEST CASE (GOOGLE SEARCH)

◇ **Step 2: Open Selenium IDE**

1. Click Selenium IDE icon
2. Click **"Create New Project"**
3. Enter Project Name → Example: **GoogleTest**

◇ **Step 3: Create New Test Case**

1. Click **"Add New Test"**
2. Name it → **TC_Google_Search_Manual**

◇ **Step 4: Record Test Steps**

👉 Click **Record Button (Red Dot)**

Now perform actions:

1. Open: https://www.google.com
2. Click search box
3. Type: **energy efficient**

4. Click outside
5. Click **Search button**

👉 Stop recording

◇ **Step 5: Add Verification Commands**

Now manually add:

Command	Target	Value
verifyText	css=h3	energy efficient
assertTitle		energy efficient - Google Search

◇ **Step 6: Save Test Case**

- Click **File → Save**
- Save as .side (or .html if required)

PART 3: AUTOMATION TEST CASE

◇ **Step 7: Create Second Test Case**

Name: **TC_Google_Search_Automation**

Steps:

1. Open Google
2. Enter: **Selenium RC**
3. Click Search
4. Verify text
5. Assert title

◇ **Commands Example:**

Command	Target	Value
open	/	
type	name=q	Selenium RC
click	name=btnK	
verifyText	css=h3	Selenium RC
assertTitle		Selenium RC - Google Search

Software Testing and Quality Assurance

PART 4: CREATE TEST SUITE

- **Step 8: Create Test Suite**
  1. Click **File → New Test Suite**
  2. Click **Add Test Cases**
  3. Add:
     - TC_Google_Search_Manual
     - TC_Google_Search_Automation
- **Step 9: Save Test Suite**
  - Save as .side / .html
- **Step 10: Execute Test Suite**
  1. Click **Run All Tests**
  2. Observe results:
     - Green → Pass ☑
     - Red → Fail ❌

OUTPUT

✓ Two test cases created
✓ Test suite created
✓ Execution successful

Practical No-7

Install Selenium WebDriver & Configure Environment (Java/Python)

Aim

To install **Selenium WebDriver** and configure the environment using **Java or Python**.

OPTION 1: JAVA SETUP (Most Common in Exams)

Tools Required

- Java JDK
- Eclipse / IntelliJ IDE
- Selenium WebDriver
- Chrome Browser

- ChromeDriver

STEP-BY-STEP PROCEDURE (JAVA)

- **Step 1: Install Java**
  1. Download Java JDK (version 8 or above)
  2. Install it
  3. Set Environment Variable:

JAVA_HOME = C:\Program Files\Java\jdk

  4. Check installation:

java -version

- **Step 2: Install Eclipse IDE**
  1. Download Eclipse
  2. Install and open
  3. Create new Java Project
- **Step 3: Download Selenium WebDriver**
  1. Go to Selenium official website
  2. Download **Selenium Java Client (.jar files)**
  3. Extract ZIP file
- **Step 4: Add Selenium to Eclipse**
  1. Right-click project → Properties
  2. Go to **Java Build Path → Libraries**
  3. Click **Add External JARs**
  4. Select all Selenium .jar files
- **Step 5: Download ChromeDriver**
  1. Go to ChromeDriver website
  2. Download version matching your Chrome
  3. Extract and copy path
- **Step 6: Set ChromeDriver Path**

In code: System.setProperty("webdriver.chrome.driver", "C:\\chromedriver.exe");

- **Step 7: Write Simple Test Program**

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class Test1 {
   public static void main(String[] args) {

       System.setProperty("webdriver.chrome.driver", "C:\\chromedriver.exe");

       WebDriver driver = new ChromeDriver();

       driver.get("https://www.google.com");

       System.out.println("Title: " + driver.getTitle());

       driver.quit();
   }
}
```

- **Step 8: Run Program**

✓ Browser should open
✓ Google page should load
✓ Title should print

OPTION 2: PYTHON SETUP (Easier)

Tools Required

- Python
- pip
- Selenium
- ChromeDriver

- **Step 1: Install Python**

Download and install Python
Check:

```
python --version
```

- **Step 2: Install Selenium**

```
pip install selenium
```

- **Step 3: Download ChromeDriver**
  - Same as Java
  - Keep in folder
- **Step 4: Write Python Script**

```
from selenium import webdriver

driver = webdriver.Chrome()

driver.get("https://www.google.com")

print(driver.title)

driver.quit()
```

- **Step 5: Run Program**

✓ Browser opens
✓ Google loads
✓ Title prints

OUTPUT

✓ Selenium installed successfully
✓ Browser automation working
✓ Title displayed

Practical No-8

Selenium WebDriver Script – Google Search Automation

Aim

To automate **Google search functionality** using Selenium WebDriver.

Objective

After completing this practical, students will be able to:
✓ Open browser using Selenium
✓ Enter search keyword
✓ Perform search action
✓ Verify results and page title

Tools Required

Software Testing and Quality Assurance

- Java / Python
- Selenium WebDriver
- Chrome Browser
- ChromeDriver

STEP-BY-STEP PROCEDURE

- **Step 1: Setup Environment**

✓ Install Java/Python
✓ Install Selenium
✓ Configure ChromeDriver

(Already done in Practical 7)

- **Step 2: Understand Steps to Automate**

Manual Steps:

1. Open browser
2. Go to Google
3. Enter search keyword
4. Click search
5. Verify results
6. Check page title

JAVA PROGRAM

- **Step 3: Write Selenium Code (Java)**

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class GoogleSearchTest {

    public static void main(String[] args) throws InterruptedException {

        // Set ChromeDriver path
        System.setProperty("webdriver.chrome.driver", "C:\\chromedriver.exe");

        // Launch browser
        WebDriver driver = new ChromeDriver();
```

```
        // Open Google
        driver.get("https://www.google.com");

        // Maximize window
        driver.manage().window().maximize();

        // Enter search keyword
        driver.findElement(By.name("q")).sendKeys("Selenium Testing");

        // Wait (for suggestion box)
        Thread.sleep(2000);

        // Click search button
        driver.findElement(By.name("btnK")).click();

        // Wait for results
        Thread.sleep(3000);

        // Get page title
        String title = driver.getTitle();

        // Verify title
        if(title.contains("Selenium Testing")) {
            System.out.println("Test Passed");
        } else {
            System.out.println("Test Failed");
        }

        // Close browser
        driver.quit();
    }
}
```

PYTHON PROGRAM

- **Step 4: Python Code**

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

# Launch browser
driver = webdriver.Chrome()
```

```
# Open Google
driver.get("https://www.google.com")

# Maximize window
driver.maximize_window()

# Enter search keyword
driver.find_element(By.NAME, "q").send_keys("Selenium Testing")

# Wait for suggestions
time.sleep(2)

# Click search
driver.find_element(By.NAME, "btnK").click()

# Wait for results
time.sleep(3)

# Verify title
if "Selenium Testing" in driver.title:
    print("Test Passed")
else:
    print("Test Failed")

# Close browser
driver.quit()
```

OUTPUT

✓ Browser opens
✓ Google loads
✓ Search executed
✓ Title verified
✓ Result displayed (Pass/Fail)

Practical No-9

Automate Form Handling (Login Page Automation using Selenium)

Aim

To automate a **login form** using Selenium WebDriver.

Objective

Software Testing and Quality Assurance

After completing this practical, students will be able to:
✓ Identify web elements (username, password, button)
✓ Perform login automation
✓ Validate login success/failure
✓ Use locators (ID, Name, XPath, CSS)

Tools Required

- Java / Python
- Selenium WebDriver
- Chrome Browser
- ChromeDriver

STEP-BY-STEP PROCEDURE

◇ **Step 1: Select Login Page**

You can use:

- Demo site: https://example.com/login
  OR
- Any sample login page

◇ **Step 2: Identify Elements (Inspect)**

Right-click → Inspect → Find:

Element	**Locator Example**
Username	name="username"
Password	name="password"

Login Button xpath="//button[@type='submit']"

◇ **Step 3: Manual Steps**

1. Open browser
2. Enter username
3. Enter password
4. Click login
5. Check result

JAVA PROGRAM

- **Step 4: Write Code (Java)**

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class LoginTest {

    public static void main(String[] args) throws InterruptedException {

        System.setProperty("webdriver.chrome.driver", "C:\\chromedriver.exe");

        WebDriver driver = new ChromeDriver();

        // Open login page
        driver.get("https://example.com/login");

        driver.manage().window().maximize();

        // Enter username
        driver.findElement(By.name("username")).sendKeys("admin");

        // Enter password
        driver.findElement(By.name("password")).sendKeys("1234");

        // Click login button
        driver.findElement(By.xpath("//button[@type='submit']")).click();

        Thread.sleep(3000);

        // Verify login (check URL or dashboard)
        String currentUrl = driver.getCurrentUrl();

        if(currentUrl.contains("dashboard")) {
            System.out.println("Login Successful");
        } else {
            System.out.println("Login Failed");
        }

        driver.quit();
    }
}
```

Software Testing and Quality Assurance

PYTHON PROGRAM

- **Step 5: Python Code**

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

driver = webdriver.Chrome()

# Open login page
driver.get("https://example.com/login")

driver.maximize_window()

# Enter username
driver.find_element(By.NAME, "username").send_keys("admin")

# Enter password
driver.find_element(By.NAME, "password").send_keys("1234")

# Click login
driver.find_element(By.XPATH, "//button[@type='submit']").click()

time.sleep(3)

# Verify login
if "dashboard" in driver.current_url:
    print("Login Successful")
else:
    print("Login Failed")

driver.quit()
```

OUTPUT

✓ Browser opens
✓ Login page loads
✓ Credentials entered
✓ Login performed
✓ Result displayed (Success/Failure)

Software Testing and Quality Assurance

Practical No-10

Handling Alerts, Frames & Multiple Windows using Selenium

Aim

To automate handling of:
✓ Alerts
✓ Frames (iFrames)
✓ Multiple browser windows/tabs

using Selenium WebDriver.

Objective

After completing this practical, students will be able to:

- Handle pop-up alerts
- Switch between frames
- Handle multiple windows/tabs
- Perform automation on dynamic web elements

Tools Required

- Java / Python
- Selenium WebDriver
- Chrome Browser
- ChromeDriver

PART 1: HANDLING ALERTS

- **Step 1: Types of Alerts**

1. Simple Alert (OK button)
2. Confirmation Alert (OK/Cancel)
3. Prompt Alert (Input box)

- **Step 2: JAVA CODE (Alerts)**

```
import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;

public class AlertHandling {
    public static void main(String[] args) throws InterruptedException {
```

```
        System.setProperty("webdriver.chrome.driver", "C:\\chromedriver.exe");

        WebDriver driver = new ChromeDriver();

        driver.get("https://demoqa.com/alerts");

        // Click alert button
        driver.findElement(By.id("alertButton")).click();

        // Switch to alert
        Alert alert = driver.switchTo().alert();

        // Print alert text
        System.out.println(alert.getText());

        // Accept alert
        alert.accept();

        driver.quit();
    }
}
```

- **Python Code**

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

driver = webdriver.Chrome()
driver.get("https://demoqa.com/alerts")

driver.find_element(By.ID, "alertButton").click()

alert = driver.switch_to.alert
print(alert.text)

alert.accept()

driver.quit()
```

Software Testing and Quality Assurance

PART 2: HANDLING FRAMES (iFrames)

◇ Step 3: What is Frame?

Frame = Web page inside another page

◇ JAVA CODE (Frames)

```
import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;

public class FrameHandling {
    public static void main(String[] args) {

        System.setProperty("webdriver.chrome.driver", "C:\\chromedriver.exe");

        WebDriver driver = new ChromeDriver();

        driver.get("https://demoqa.com/frames");

        // Switch to frame
        driver.switchTo().frame("frame1");

        // Print text inside frame
        String text = driver.findElement(By.id("sampleHeading")).getText();
        System.out.println(text);

        // Switch back
        driver.switchTo().defaultContent();

        driver.quit();
    }
}
```

◇ Python Code

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.get("https://demoqa.com/frames")

driver.switch_to.frame("frame1")
```

```
print(driver.find_element(By.ID, "sampleHeading").text)

driver.switch_to.default_content()

driver.quit()
```

PART 3: HANDLING MULTIPLE WINDOWS

- **Step 4: Concept**

When clicking link → new window/tab opens

- **JAVA CODE (Multiple Windows)**

```
import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;
import java.util.Set;

public class WindowHandling {
    public static void main(String[] args) {

        System.setProperty("webdriver.chrome.driver", "C:\\chromedriver.exe");

        WebDriver driver = new ChromeDriver();

        driver.get("https://demoqa.com/browser-windows");

        // Click new window button
        driver.findElement(By.id("windowButton")).click();

        // Get all windows
        Set<String> windows = driver.getWindowHandles();

        for(String window : windows) {
            driver.switchTo().window(window);
        }

        System.out.println(driver.getTitle());

        driver.quit();
    }
}
```

Software Testing and Quality Assurance

- **Python Code**

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.get("https://demoqa.com/browser-windows")

driver.find_element(By.ID, "windowButton").click()

windows = driver.window_handles

for window in windows:
    driver.switch_to.window(window)

print(driver.title)

driver.quit()
```

OUTPUT

✓ Alert handled successfully
✓ Frame accessed correctly
✓ Multiple windows switched